VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science Engineering and Information Systems

BCSE302L Database Systems

Course Project

BOM-Track

A Database for Hardware Bills of Materials, Part Stock and Board Builds

Review III: Integrated System and Final Demonstration

Submitted by	: Vishwak
Register Number	: 24BYB0053
Implementation	: PostgreSQL 16 (SQL, PL/pgSQL), Python 3, Streamlit
Faculty	:

October 2026

Contents

1. Project Title and Student Details
2. Introduction and Problem Statement
3. Functional Requirements
4. ER Diagram
5. Relational Schema
6. Table Creation and Constraints
7. SQL Queries and Outputs
8. Procedures, Functions, Cursor and Triggers
9. Real-World Transaction Demonstrations
10. Conclusion
Appendix A. How to Run

1. Project Title and Student Details

Item	Detail
Title	BOM-Track: A Database for Hardware Bills of Materials, Part Stock and Board Builds
Student	Vishwak (individual project)
Register Number	24BYB0053
Course	BCSE302L Database Systems
Platform	PostgreSQL 16 with PL/pgSQL; Python 3 with psycopg2 and Streamlit

2. Introduction and Problem Statement

When I design a PCB in KiCad, the bill of materials (BOM) comes out as a CSV file. For my 8051 board, uGameBox, it has 28 lines that group 49 components. The file is a parts list and nothing more: it cannot tell me what the board costs, what I still need to buy, or whether I have enough parts to build it. Those answers are usually worked out by hand and go stale as soon as a price changes or parts are used.

The file also mixes stored facts with numbers that can be calculated. Refs holds a list of designators in one cell, Qty is just a count of that list, and Value is sometimes a label (nine identical push buttons appear as nine lines named BTN0 to BTN7 and RESET).

BOM-Track stores only basic facts: which part is at which position on which board, which price a supplier quoted and when, how many parts came in, and when a build was started and finished. Everything else (quantities, cost, stock on hand, shortfall, readiness) is calculated by views and PL/pgSQL code each time it is needed. The database also enforces the rules of the domain with constraints and triggers, so the application cannot put it into an impossible state.

3. Functional Requirements

ID	Requirement	Implemented by
FR1	Import a KiCad BOM, one placement per designator	bomlib.import_board (one transaction); app page Import
FR2	Single parts catalogue with categories	part, category; UNIQUE(category_id, value_text, footprint)
FR3	Boards and their placements	board, placement; UNIQUE(board_id, ref_designator)
FR4	Suppliers and dated price quotes	supplier, price_quote; v_latest_quote
FR5	Purchases and adjustments	sp_record_purchase; stock_movement; trg_stock_guard
FR6	Plan and complete builds	build; sp_complete_build; trg_build_complete
FR7	Cost, stock, shortfall, readiness, shared parts	v_board_cost, v_stock_on_hand, v_build_shortfall, v_build_readiness, v_shared_parts, fn_board_cost
FR8	Reject invalid operations	CHECK and FK constraints; 3 triggers

4. ER Diagram

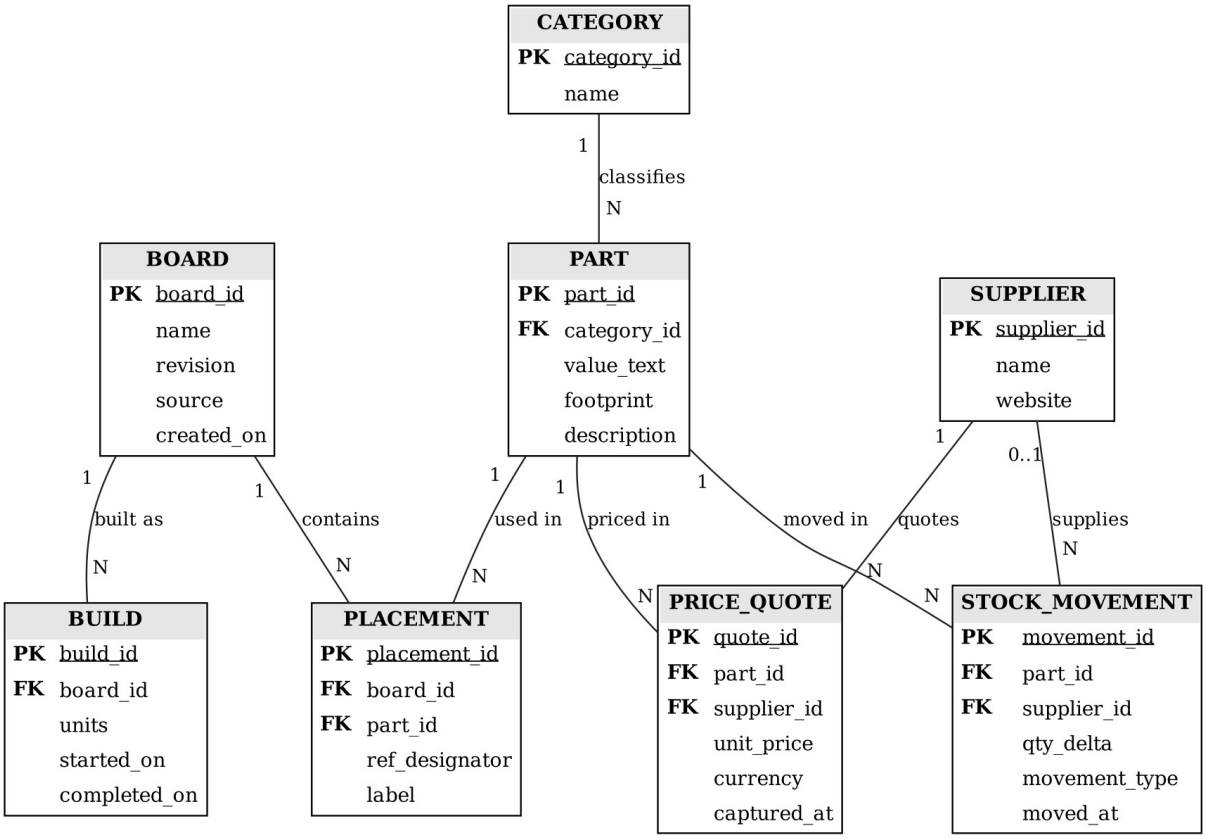


Figure 1. ER diagram (PK = primary key, FK = foreign key)

BOARD and PART are in an M:N relationship resolved by PLACEMENT, which also carries the designator and label. PART and SUPPLIER are M:N through PRICE_QUOTE. Every other relationship is 1:N. A stock movement has zero or one supplier: a purchase names one, an adjustment does not.

5. Relational Schema

*Primary keys are underlined, foreign keys are marked with *.*

CATEGORY (category_id, name **)**

PART (part_id, category_id*, value_text, footprint, description **)**

BOARD (board_id, name, revision, source, created_on **)**

PLACEMENT (placement_id, board_id*, part_id*, ref_designator, label **)**

SUPPLIER (supplier_id, name, website **)**

PRICE_QUOTE (quote_id, part_id*, supplier_id*, unit_price, currency, captured_at **)**

BUILD (build_id, board_id*, units, started_on, completed_on **)**

STOCK_MOVEMENT (movement_id, part_id*, supplier_id*, qty_delta, movement_type, moved_at **)**

All eight tables are in BCNF: in each table every determinant is a candidate key (the surrogate key, plus the declared natural keys such as (board_id, ref_designator) and (category_id, value_text, footprint)). The functional dependencies are listed in the Review I document.

6. Table Creation and Constraints

The full DDL is in sql/01_schema.sql and is listed in the Review II document. Summary of the constraints:

Type	Count	Purpose
PRIMARY KEY	8	Surrogate key in every table
FOREIGN KEY	8	Every reference between tables; placement -> board cascades on delete
UNIQUE	5	Natural keys: part identity, designator per board, board name + revision, category and supplier names
CHECK	6	Positive prices and units, valid dates, non-zero movements, movement type, purchase rule
NOT NULL	31 columns	Keys, names, values and dates
DEFAULT	7 columns	INR currency, current date/time, 1 unit

Data loaded from real sources: 11 boards (my uGameBox and 10 published open-hardware designs), 242 parts, 678 placements, 15 categories, 11 suppliers and 23 dated price quotes (18 Mouser prices from 2010 in USD, 5 Probots/Zbotic prices from 14 October 2026 in INR).

7. SQL Queries and Outputs

Seventeen queries were run (full SQL and output in Review II and logs/queries_output.log):

#	Question answered	Rows
Q1	All boards with their source and number of placed components (JOIN + GROUP BY)	11
Q2	Full BOM of uGameBox, grouped like a KiCad export (view v_board_bom)	20
Q3	Placements that carry a label different from the part value (uGameBox buttons)	9
Q4	Number of distinct parts and placements per category (aggregation)	15
Q5	Most reused parts across all boards (v_shared_parts, top 10)	10

Q6	Values on uGameBox that other boards also use, but in a different package (subquery with IN)	4
Q7	Parts that are on uGameBox and on no other board (NOT EXISTS)	20
Q8	Boards bigger than the average board (HAVING with a scalar subquery)	6
Q9	Board cost from latest quotes, with price coverage (v_board_cost)	8
Q10	Cost of uGameBox through function fn_board_cost	1
Q11	Supplier comparison: number of quotes and average unit price (OUTER JOIN)	11
Q12	Supplier ranking per part: cheapest quote first (window function RANK)	5
Q13	Unpriced lines of uGameBox (anti join)	16
Q14	Capacitor values used on the Ubertooth family, by package (string functions + CASE)	6
Q15	Pairs of boards that share the most parts (self join)	8
Q16	Where-used: every board and designator that uses a 100nF 0402 capacitor (correlated lookup)	5
Q17	Number of placements per designator prefix (regular expressions)	11

Sample outputs

Q2. Full BOM of uGameBox, grouped like a KiCad export (view v_board_bom)

```
                refs                  |   category    |     value_text      | qty
--------------------------------------+---------------+---------------------+-----
 BZ1                                  | Buzzer        | Active 5V           |   1
 C1,C6,C7                             | Capacitor     | 100nF               |   3
 C2,C3                                | Capacitor     | 10uF                |   2
 C4,C5                                | Capacitor     | 33pF                |   2
 J1                                   | Connector     | 9V BATT             |   1
 J3                                   | Connector     | ISP 2x5             |   1
 J2                                   | Connector     | SW-420 (VCC/GND/DO) |   1
 Y1                                   | Crystal       | 11.0592MHz          |   1
 D10,D9                               | Diode         | 1N5819              |   2
 DS1                                  | Display       | LCD 16x2 (HD44780)  |   1
 U1                                   | IC / Module   | AT89S52             |   1
 U2                                   | IC / Module   | L7805               |   1
...
(20 rows)
```

Q9. Board cost from latest quotes, with price coverage (v_board_cost)

```
        board        |  revision   | currency | priced_cost | lines_priced
---------------------+-------------+----------+-------------+--------------
 uGameBox            | rev 1       | INR      |      315.84 | 4 of 20
 Ubertooth One       | git c9dfdbd | USD      |       11.47 | 14 of 49
 Ubertooth Celery    | git c9dfdbd | USD      |       10.61 | 8 of 56
 TC13 Badge          | git c9dfdbd | USD      |       10.19 | 12 of 57
 Ubertooth Artichoke | git c9dfdbd | USD      |        1.12 | 9 of 37
 Ubertooth Broccoli  | git c9dfdbd | USD      |        1.03 | 6 of 46
 Ubertooth Pogoprog  | git c9dfdbd | USD      |        0.12 | 2 of 8
 Ubertooth Endive    | git c9dfdbd | USD      |        0.09 | 1 of 10
(8 rows)
```

Q15. Pairs of boards that share the most parts (self join)

```
             board_a              |             board_b              | common_parts
----------------------------------+----------------------------------+--------------
 Ubertooth Broccoli git c9dfdbd  | Ubertooth Celery git c9dfdbd     |           39
 Ubertooth Celery git c9dfdbd    | TC13 Badge git c9dfdbd           |           33
 Ubertooth Broccoli git c9dfdbd  | TC13 Badge git c9dfdbd           |           32
 Ubertooth One git c9dfdbd       | Ubertooth Artichoke git c9dfdbd |           26
 OtterPill v1.3                  | OtterPill v1.4                   |           24
 Ubertooth One git c9dfdbd       | TC13 Badge git c9dfdbd           |           19
 Ubertooth One git c9dfdbd       | Ubertooth Celery git c9dfdbd     |           15
 Ubertooth Artichoke git c9dfdbd | TC13 Badge git c9dfdbd           |           14
(8 rows)
```

8. Procedures, Functions, Cursor and Triggers

Component	Type	Used in	What it does
`fn_stock_on_hand`	Function	Stock trigger, app	Stock on hand of one part, from movements minus consumption
`fn_board_cost`	Function	Board page, Q10	Cost of a board in a currency from latest quotes
`sp_record_purchase`	Procedure	Purchase page	Inserts a PURCHASE movement; stores the paid price as a quote
`sp_complete_build`	Procedure	Builds page	Validates and completes a build
`sp_shopping_list`	Procedure with cursor	Builds page	Cursor over the shortfall: one line per part, total cost, unpriced count
`trg_stock_guard`	Trigger (BEFORE INSERT)	Every stock movement	Stops stock going below zero; locks the part row
`trg_build_complete`	Trigger (BEFORE UPDATE)	Completing a build	Rejects completion if any part is short; locks the board's parts
`trg_freeze_bom`	Trigger (BEFORE I/U/D)	Editing placements	Blocks BOM edits on boards that have been built

9. Real-World Transaction Demonstrations

The flow is the same for every transaction: the user acts in the Streamlit app, the app runs SQL or calls a PL/pgSQL procedure through psycopg2, PostgreSQL applies constraints and triggers, commits or rolls back, and the page is redrawn from the views. Each screen shows the SQL that was executed. The screenshots were taken from the running app in a browser.

Important: these transactions are demonstrations. The purchases recorded here did not really happen; they exist only to exercise the system. The script sql/99_reset_demo.sql removes them and returns the database to the real data.

```
# app.py: every write runs in its own transaction
def run(sql, params=None):
    c = conn(); c.notices.clear()
    try:
        with c:                      # psycopg2: COMMIT on success, ROLLBACK on exception
            with c.cursor() as cur:
                cur.execute(sql, params)
                out = cur.fetchall() if cur.description else None
        return True, out, [n.strip() for n in c.notices]
    except psycopg2.Error as e:
        return False, e.diag.message_primary, []
```

9.1 Transaction 1: Import a KiCad BOM file

Question	Answer
Operation	The user uploads a KiCad BOM CSV and gives a board name and revision.
Tables involved	board, category, part, placement
SQL / PL/pgSQL executed	Python importer (bomlib.import_board) running INSERTs in one transaction; UNIQUE constraints decide whether a part already exists.
Effect on the database	One board row and 49 placement rows are added. Parts that already exist are reused, so no part is duplicated. If any line fails, the whole board is rolled back.
Result shown	"Board created with 49 placements. Skipped: none".

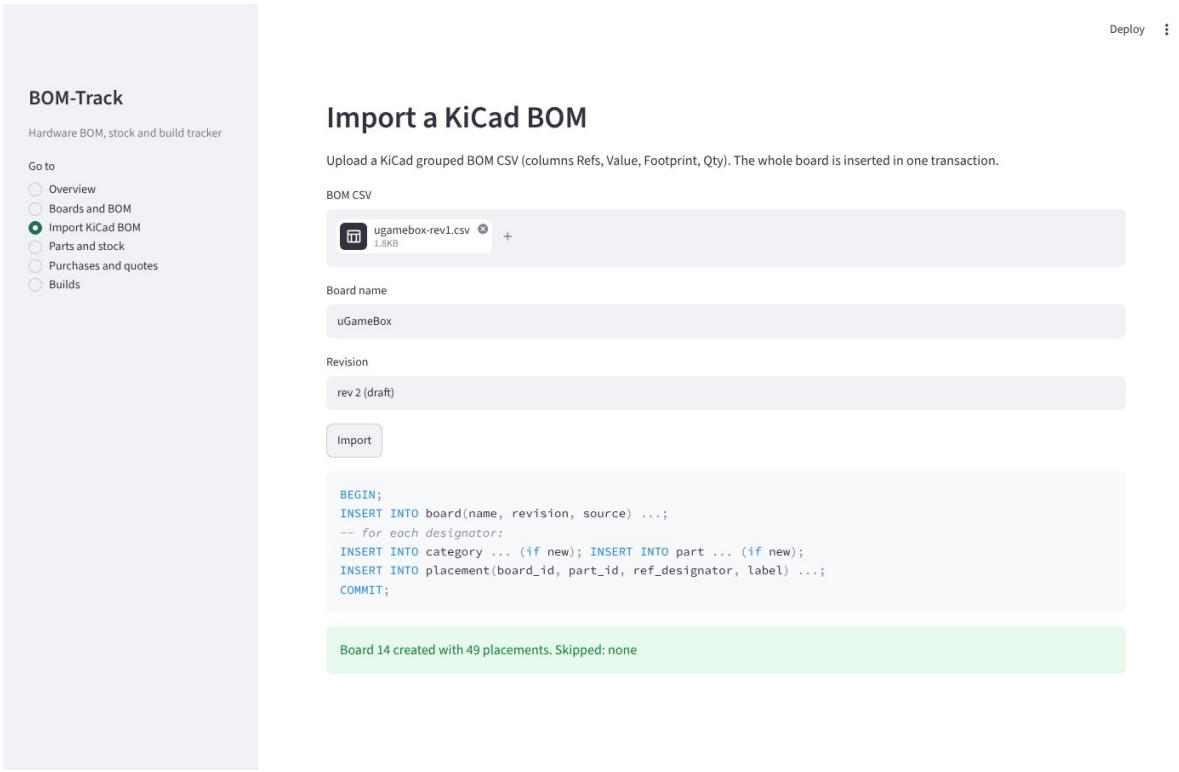


Figure 2. Import a KiCad BOM file

9.2 Transaction 2: Complete a build with no parts in stock

Question	Answer
Operation	A build of 1 uGameBox is planned (INSERT into build), then the user clicks Complete build.
Tables involved	build (update attempted); read: placement, stock_movement via v_build_shortfall
SQL / PL/pgSQL executed	CALL sp_complete_build(1) -> UPDATE build -> trigger trg_build_complete.
Effect on the database	Nothing changes. The trigger finds parts short and raises an exception; the transaction is rolled back.
Result shown	"Rejected by the database: build 1 cannot be completed: short of 9 x Resistor 1k".

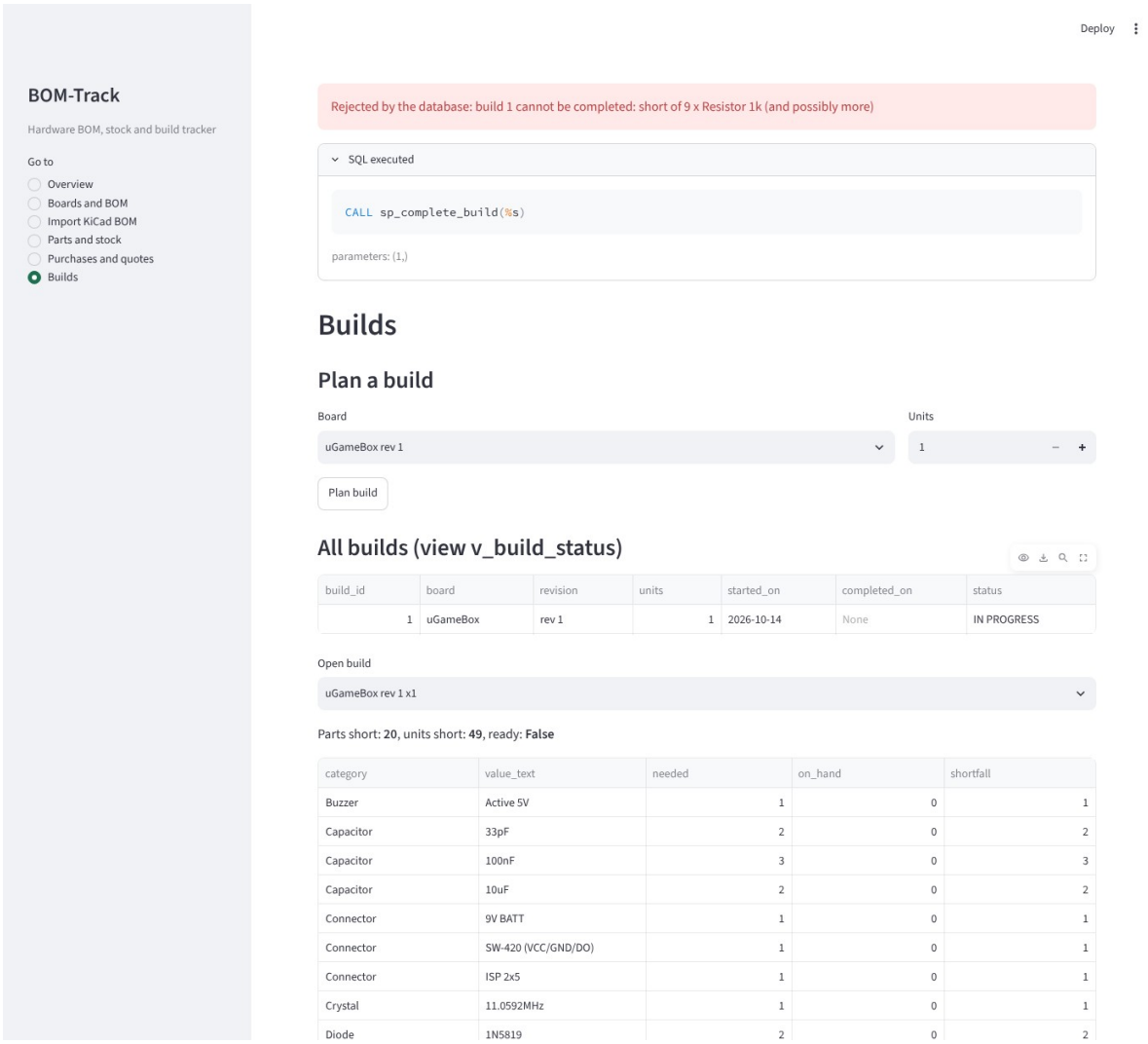


Figure 3. Complete a build with no parts in stock

9.3 Transaction 3: Shopping list for the build (cursor)

Question	Answer
Operation	The user asks for a shopping list for the open build.
Tables involved	Read only: v_build_shortfall, v_latest_quote, supplier
SQL / PL/pgSQL executed	CALL sp_shopping_list(1, 'INR', NULL, NULL): explicit cursor, FETCH loop, RAISE NOTICE per part, INOUT totals.
Effect on the database	No change.
Result shown	20 lines, one per short part. INR 315.84 for the 4 priced lines; 16 lines have no INR quote yet.

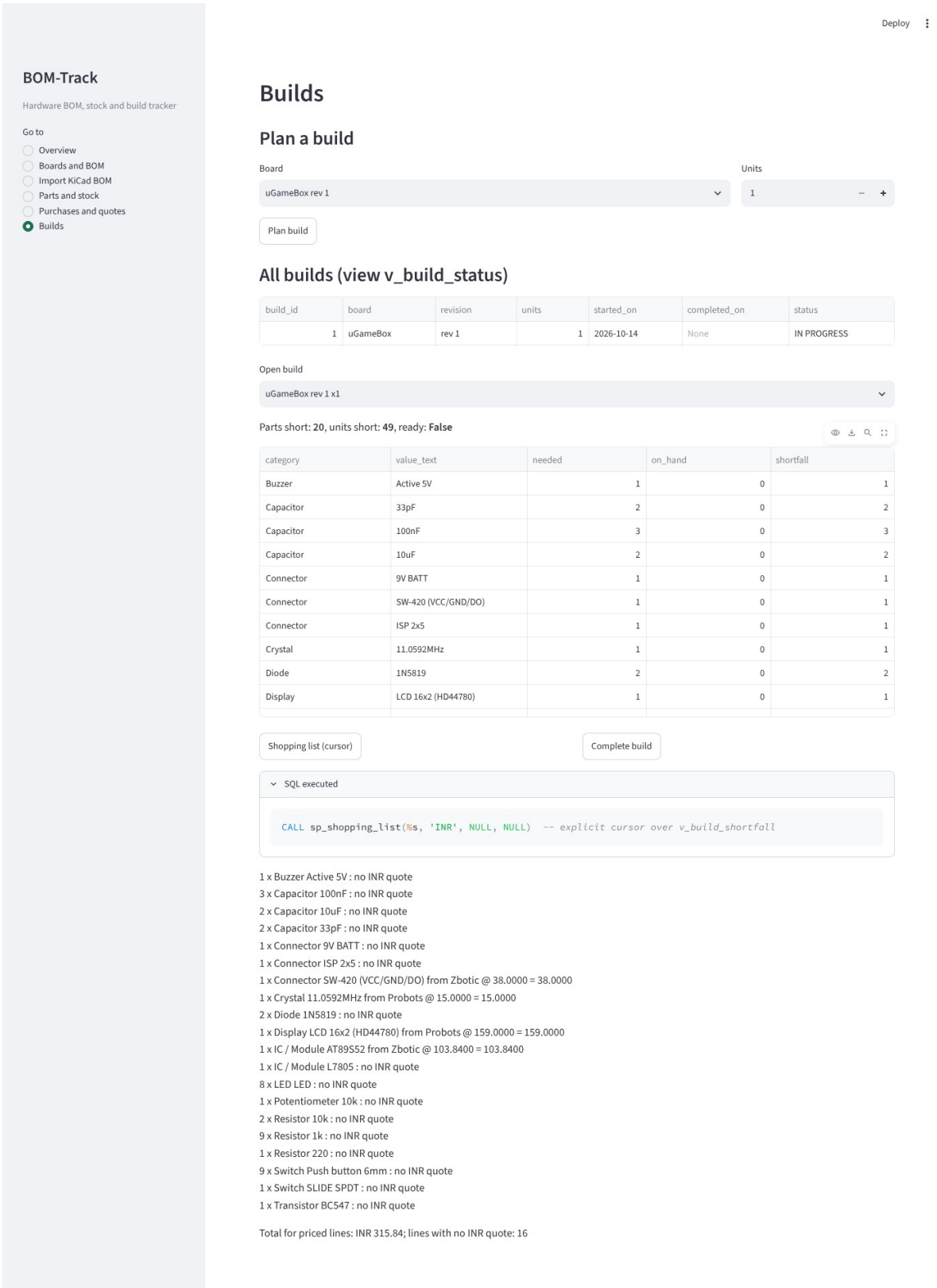


Figure 4. Shopping list for the build (cursor)

9.4 Transaction 4: Record a purchase

Question	Answer
Operation	The user records buying 1 AT89S52 from Zbotic at INR 103.84.
Tables involved	stock_movement, price_quote
SQL / PL/pgSQL executed	CALL sp_record_purchase(18, 11, 1, 103.84, 'INR'); trigger trg_stock_guard runs on the insert.
Effect on the database	One PURCHASE row in stock_movement (+1) and one row in price_quote. Stock on hand rises through v_stock_on_hand.
Result shown	"Purchase recorded".

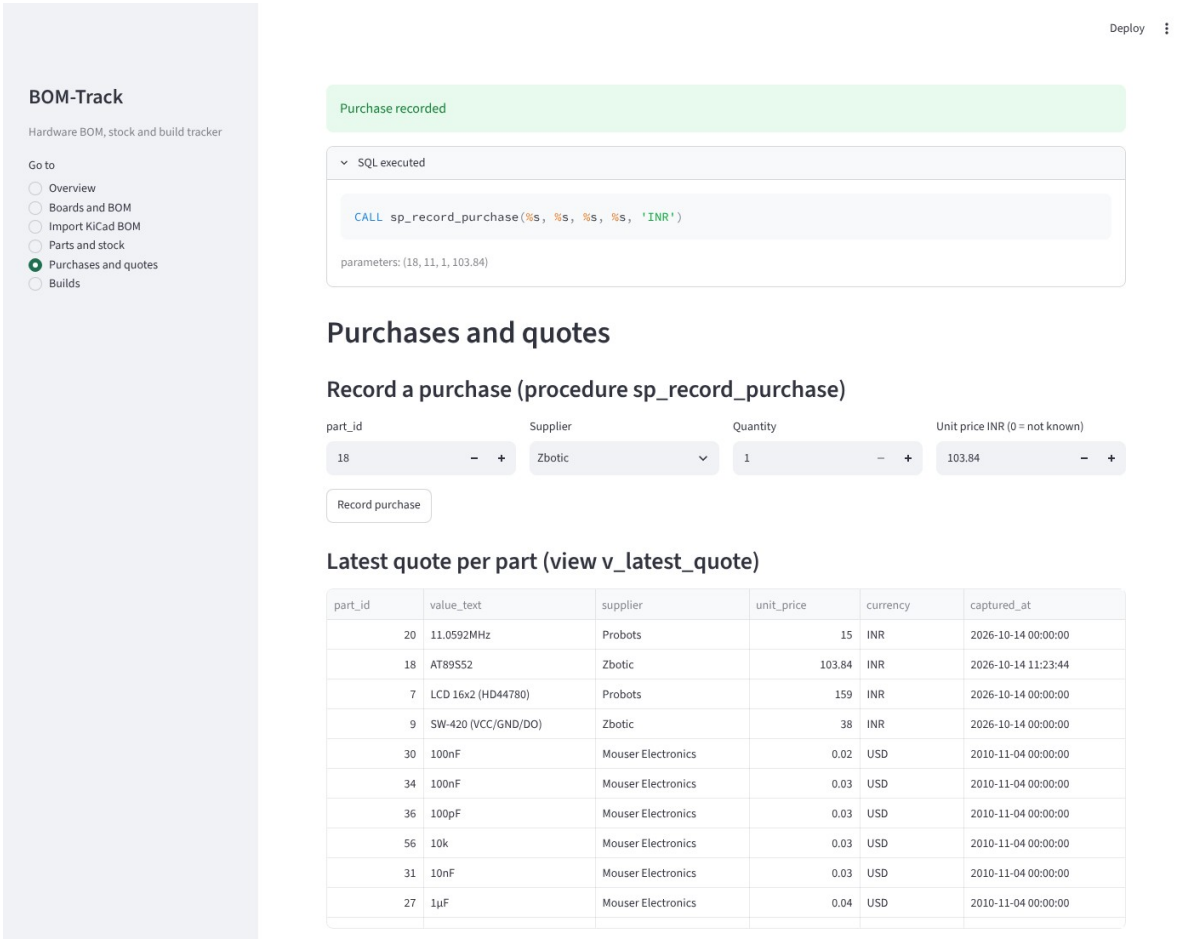


Figure 5. Record a purchase

9.5 Transaction 5: Complete the build once stock is available

Question	Answer
Operation	After the remaining shortfall is bought (same procedure, called in a loop), the user clicks Complete build again.
Tables involved	build; read: placement, stock_movement
SQL / PL/pgSQL executed	CALL sp_complete_build(1); trigger trg_build_complete locks the parts and finds no shortfall.
Effect on the database	completed_on is set. The status in v_build_status changes to COMPLETED and the 49 components are subtracted in v_stock_on_hand through v_part_consumed. No stock column is updated, because none exists.
Result shown	"Build 1 completed; its parts are now consumed from stock".

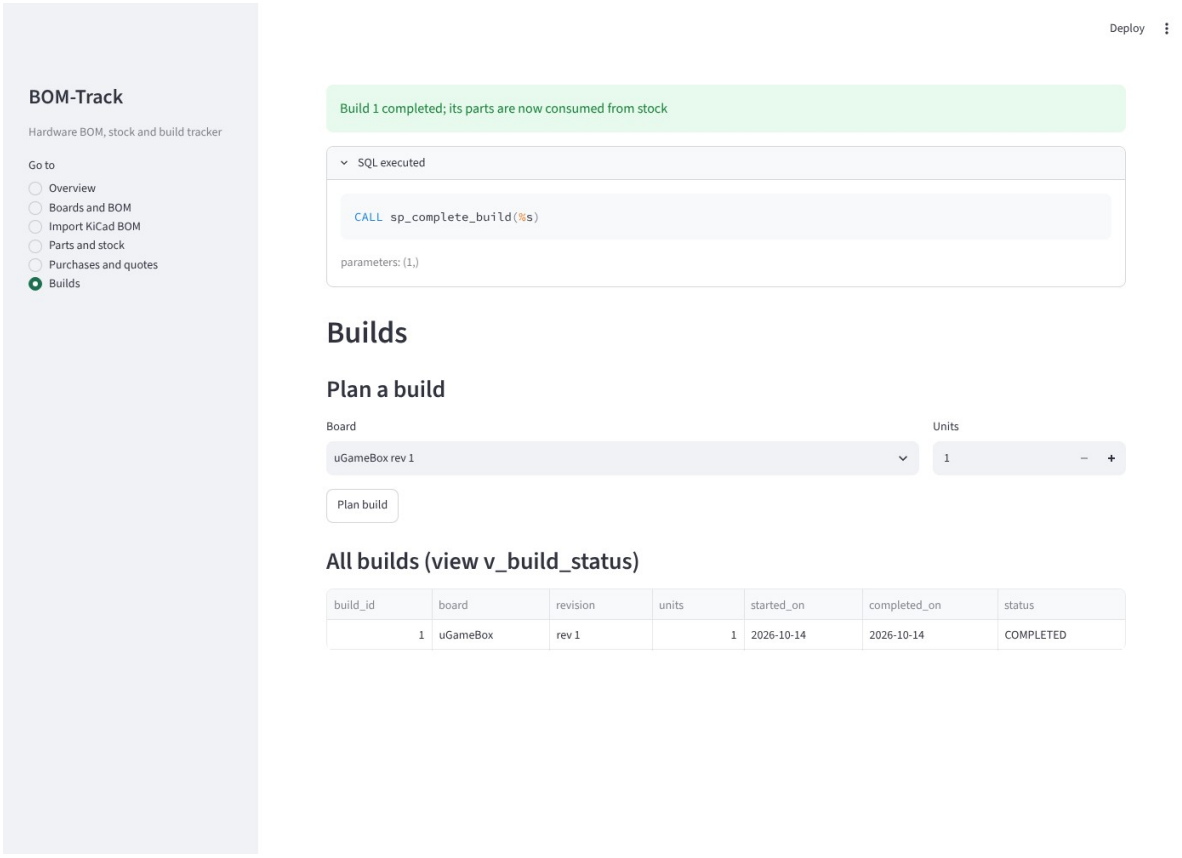


Figure 6. Complete the build once stock is available

9.6 Transaction 6: Write off a damaged LED when none is left

Question	Answer
Operation	The user records -1 LED as an adjustment. All 8 LEDs were used by the build.
Tables involved	stock_movement (insert attempted)
SQL / PL/pgSQL executed	INSERT into stock_movement -> trigger trg_stock_guard -> function fn_stock_on_hand.
Effect on the database	Nothing changes; the insert is rejected.
Result shown	"stock for part 5 would go negative (on hand 0, change -1)".

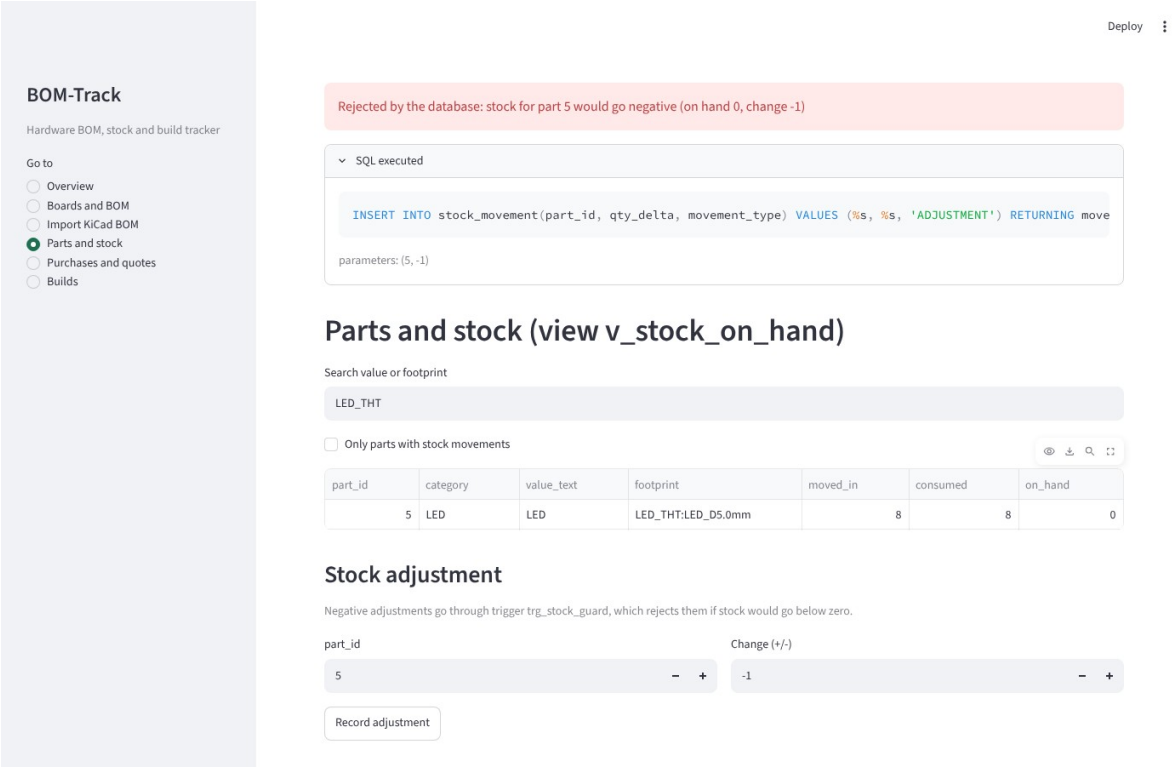


Figure 7. Write off a damaged LED when none is left

9.7 Transaction 7: Editing the BOM of a board that has been built

Question	Answer
Operation	Delete R11 from uGameBox, and relabel SW1, after build 1 was completed.
Tables involved	placement (delete and update attempted); read: build
SQL / PL/pgSQL executed	DELETE / UPDATE on placement -> trigger trg_freeze_bom
Effect on the database	Nothing changes. Past consumption is computed from the placements, so changing them would rewrite history.
Result shown	"board 1 has completed builds; create a new revision instead of editing its BOM"

```
DELETE FROM placement WHERE board_id = 1 AND ref_designator = 'R11';
ERROR:  board 1 has completed builds; create a new revision instead of editing its BOM
UPDATE placement SET label = 'FIRE' WHERE board_id = 1 AND ref_designator = 'SW1';
ERROR:  board 1 has completed builds; create a new revision instead of editing its BOM
```

9.8 Transaction 8: Savepoint inside one transaction

A purchase is recorded, then a quote with a negative price is attempted inside the same transaction. ROLLBACK TO SAVEPOINT undoes only the bad step; the purchase is kept when the transaction commits.

```
BEGIN;
BEGIN
CALL sp_record_purchase(12, 5, 20);
CALL
SAVEPOINT before_bad;
SAVEPOINT
INSERT INTO price_quote(part_id, supplier_id, unit_price) SELECT part_id, 1, -5 FROM placement
WHERE board_id = 1 AND ref_designator = 'R1';
```

```
ERROR:  new row for relation "price_quote" violates check constraint
"price_quote_unit_price_check"
ROLLBACK TO SAVEPOINT before_bad;
ROLLBACK
COMMIT;
COMMIT
SELECT value_text, moved_in, consumed, on_hand FROM v_stock_on_hand
WHERE part_id = (SELECT part_id FROM placement WHERE board_id = 1 AND ref_designator = 'R1');
 value_text | moved_in | consumed | on_hand
------------+----------+----------+---------
 1k         |       29 |        9 |      20
(1 row)
```

9.10 Concurrency control

Two sessions try to complete two different uGameBox builds at the same moment, with stock for only one board. Each check reads stock that the other session has not committed yet. To make the timing repeatable, a delay can be set for the demo session (setting bomtrack.demo_delay = 3 seconds); it is not used in normal operation.

Run 1 uses a version of the trigger without row locks. Both sessions pass the check and both builds complete:

```
build_id |  status
----------+-----------
        1 | COMPLETED
        2 | COMPLETED
(2 rows)

   value_text    | moved_in | consumed | on_hand
-----------------+----------+----------+---------
 1k              |        9 |       18 |      -9
 Push button 6mm |        9 |       18 |      -9
 LED             |        8 |       16 |      -8
 100nF           |        3 |        6 |      -3
 33pF            |        2 |        4 |      -2
(5 rows)
```

Stock went negative (-9 push buttons, -9 resistors, -8 LEDs). This is a write-skew anomaly: each transaction was correct on its own, but together they used the same parts twice.

Run 2 uses the real trigger, which takes row locks on the board's parts with SELECT ... FOR UPDATE before checking:

```
[A]       t       |          event
[A] --------------+-------------------------
[A]  11:25:26.196 | start: complete build 1
[A]
[A]       t       | event
[A] --------------+-------
[A]  11:25:29.214 | end
[A]
[B]       t       |          event
[B] --------------+-------------------------
[B]  11:25:27.204 | start: complete build 2
[B]
[B] ERROR:  build 2 cannot be completed: short of 9 x Switch Push button 6mm (and possibly
more)
[B]       t       | event
[B] --------------+-------
[B]  11:25:32.223 | end
[B]
```

```
 build_id |   status
----------+-------------
        1 | COMPLETED
        2 | IN PROGRESS
(2 rows)
 value_text | moved_in | consumed | on_hand
------------+----------+----------+---------
 1k         |        9 |        9 |       0
 AT89S52    |        1 |        1 |       0
(2 rows)
```

Session B started at 11:25:27.2 but was blocked on the row lock until A committed at 11:25:29.2. It then ran its own check, saw that A had used the parts, and was rejected (it ends at 32.2 because of the 3-second demo delay). Stock ends at exactly 0. Under READ COMMITTED, each statement in the PL/pgSQL trigger takes a fresh snapshot, so the check after the lock sees A's committed change.

9.11 Crash recovery

One transaction commits a purchase of 5 BC547. A second transaction inserts a purchase of 7 L7805 and is still open when the server is stopped with pg_ctl stop -m immediate, which kills it without a checkpoint, like a power cut.

```
--- 1. Committed transaction: buy 5 x BC547
BEGIN
DO
COMMIT
--- 2. Open transaction (never committed): buy 7 x L7805, then the server is killed
         note         | rows_visible
----------------------+--------------
 from another session |            1
(1 row)

--- 3. Kill the server: pg_ctl stop -m immediate (like a power cut, no checkpoint)
server stopped
[open session output]
BEGIN
INSERT 0 1
          note           | rows_visible_here
-------------------------+-------------------
 inside open transaction |                 2
(1 row)

WARNING:  terminating connection due to immediate shutdown command
server closed the connection unexpectedly
        This probably means the server terminated abnormally
        before or while processing the request.
connection to server was lost
--- 4. Restart
waiting for server to start.... done
server started
[server log during startup]
LOG:  database system was interrupted; last known up at 2026-10-14 11:25:20 UTC
LOG:  database system was not properly shut down; automatic recovery in progress
LOG:  redo starts at 0/37AB570
CONTEXT:  WAL redo at 0/37AB620 for Database/DROP: dir 1663/41155
CONTEXT:  WAL redo at 0/37AB620 for Database/DROP: dir 1663/41155
LOG:  redo done at 0/3C7CA50 system usage: CPU: user: 0.01 s, system: 0.01 s, elapsed: 0.03 s
LOG:  checkpoint complete: wrote 1026 buffers (6.3%); 0 WAL file(s) added, 0 removed, 0 recycled;
write=0.014 s, sync=0.037 s, total=0.055 s; sync files=338, longest=0.010 s, average=0.001 s;
distance=4933 kB, estimate=4933 kB; lsn=0/3C7CA78, redo lsn=0/3C7CA78
LOG:  database system is ready to accept connections
--- 5. What survived
 value_text | qty_delta | movement_type
------------+-----------+---------------
 BC547      |         5 | PURCHASE
```

```
(1 row)
```

On restart PostgreSQL detects the unclean shutdown and replays the write-ahead log (redo). The committed purchase is present (durability) and the uncommitted one is gone (atomicity). While the second transaction was open, another session saw only 1 row and the open session saw 2 (isolation).

9.12 Indexing

```
=== 1. Size of the real tables
   table_name   | rows | heap_size  | pages
----------------+------+------------+-------
 part           |    0 | 24 kB      |     3
 supplier       |    0 | 8192 bytes |     1
 build          |    0 | 0 bytes    |     0
 placement      |    0 | 32 kB      |     4
 stock_movement |    0 | 0 bytes    |     0
 category       |    0 | 8192 bytes |     1
 price_quote    |    0 | 8192 bytes |     1
 board          |    0 | 8192 bytes |     1
(8 rows)
```

The where-used query (all placements of one part) without an index on placement(part_id), and then with it:

```
                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------
--------
 Seq Scan on placement  (cost=0.00..12.47 rows=37 width=7) (actual time=0.012..0.057 rows=37
loops=1)
   Filter: (part_id = 34)
   Rows Removed by Filter: 641
   Buffers: shared hit=4
 Planning:
   Buffers: shared hit=32
 Planning Time: 0.105 ms
 Execution Time: 0.065 ms
(8 rows)
```

```
                                                              QUERY PLAN
-----------------------------------------------------------------------------------------------------
------------------------------
 Bitmap Heap Scan on placement  (cost=4.56..9.02 rows=37 width=7) (actual time=0.050..0.059
rows=37 loops=1)
   Recheck Cond: (part_id = 34)
   Heap Blocks: exact=3
   Buffers: shared hit=3 read=2
   ->  Bitmap Index Scan on idx_placement_part  (cost=0.00..4.55 rows=37 width=0) (actual
time=0.039..0.039 rows=37 loops=1)
         Index Cond: (part_id = 34)
         Buffers: shared read=2
 Planning:
   Buffers: shared hit=17 read=1
 Planning Time: 0.211 ms
 Execution Time: 0.079 ms
(11 rows)
```

With the index the planner switches from a sequential scan to a bitmap index scan, and its cost estimate drops from 12.47 to 9.02. But the measured times do not show a gain: 0.065 ms without the index, 0.079 ms with it on its first run (the index pages had to be read from disk, shown as

"read=2" in the buffers line) and 0.028 ms when repeated. These differences are noise. The whole placement table is 678 rows in 4 pages of 8 KB, so reading all of it is already instant. I did not create a large artificial table to show a bigger speed-up, because the real data does not need one. The index is kept because it is the correct design for the join and will matter as more boards are imported. Lookups by designator already use the index created for UNIQUE(board_id, ref_designator):

```
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------
-------------------------
 Index Scan using uq_placement on placement  (cost=0.28..8.29 rows=1 width=4) (actual
time=0.013..0.013 rows=1 loops=1)
   Index Cond: ((board_id = 1) AND ((ref_designator)::text = 'U1'::text))
 Planning Time: 0.042 ms
 Execution Time: 0.020 ms
(4 rows)
```

9.13 Reports produced by the system

The overview page reads the cost and readiness views directly:

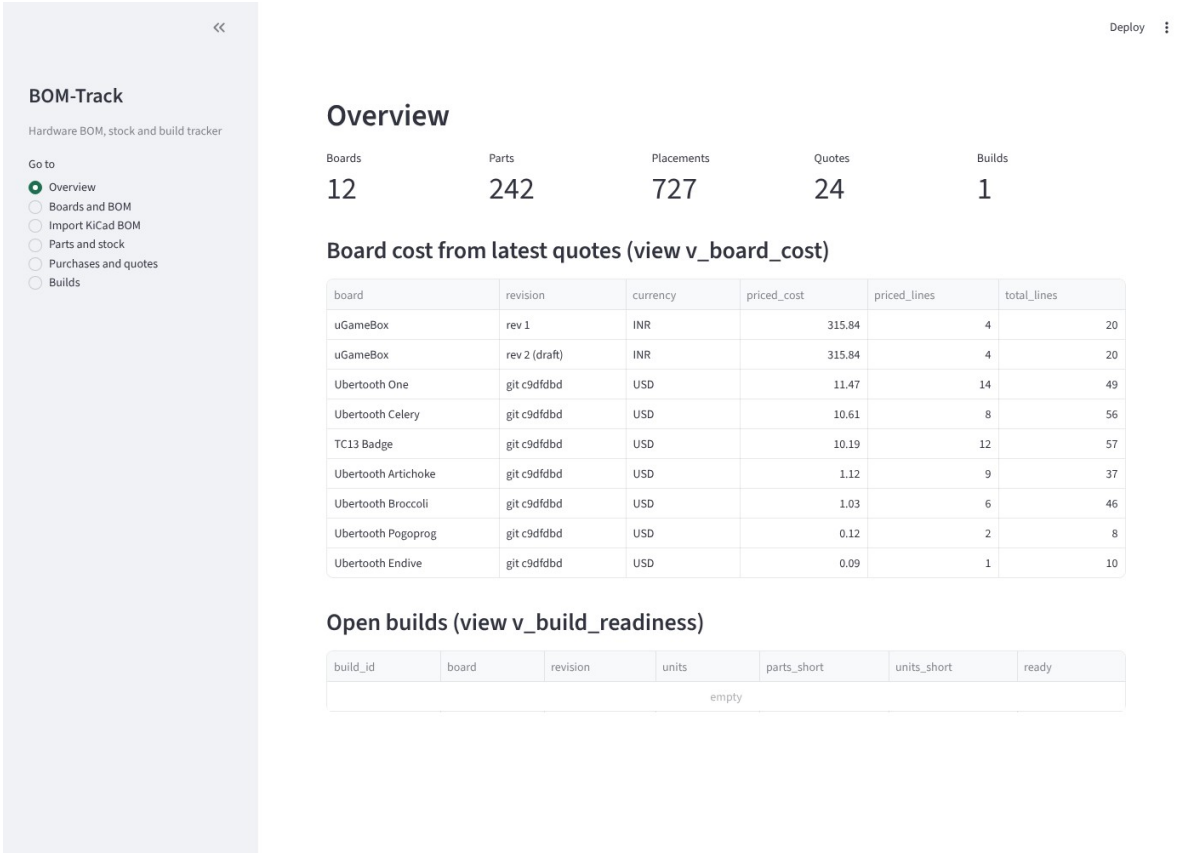


Figure 8. Overview after the demonstration

10. Conclusion

BOM-Track turns flat KiCad BOM exports into a normalized PostgreSQL database in which the database does the work: quantities, costs, stock, shortfall and build status are all computed by views, functions and procedures, and the rules of the domain are enforced by constraints and triggers. Every component listed in the guidelines is used inside a real operation of the application, and every output in this report was produced by running the code.

10.1 Limitations

- Purchases, builds and stock movements in the demonstrations are not real; I have not bought parts for this project yet.
- Price coverage is partial: 4 of 20 uGameBox lines have an INR price, and the USD prices for the Ubertooth boards are from 2010.
- A part is identified by its exact value and footprint text, so "100nF" and "100n" are stored as different parts. Normalizing values is future work.
- Categories are assigned by rules on the designator prefix and symbol name; unusual designators could be misclassified.
- The shortfall of each open build is checked against current stock on its own, so two open builds may both look ready while competing for the same parts. The trigger still prevents both from being completed.
- The data set is small (678 placements), so indexing changes the query plan but not the measured time.

10.2 Future work

- Normalize component values (100n = 100nF = 0.1uF) before matching parts.
- Import LCSC part numbers where the BOM has them and fetch prices automatically.
- Reserve stock for planned builds so that open builds do not compete silently.

Appendix A. How to Run

```
# Windows (PostgreSQL 18 installed at the default path)
pip install -r requirements.txt
set PGPASSWORD=<your postgres password>
build_db.bat
python -m streamlit run app.py          # opens http://localhost:8501

# Linux / macOS
./build_db.sh
./run_all_demos.sh                      # regenerates every file in logs/
python3 -m streamlit run app.py
```